PROJECT OVERVIEW

# Patungan — Split Bill

A split bill app for groups of friends: record shared bills, split them equally or by item, and see exactly who should pay whom.

Raihan Evanza · Fullstack & Mobile Developer · portfolio-raihan-evanza.vercel.app

**ROLE**

Personal project · design, Go backend, React web, Flutter app, demo deployment

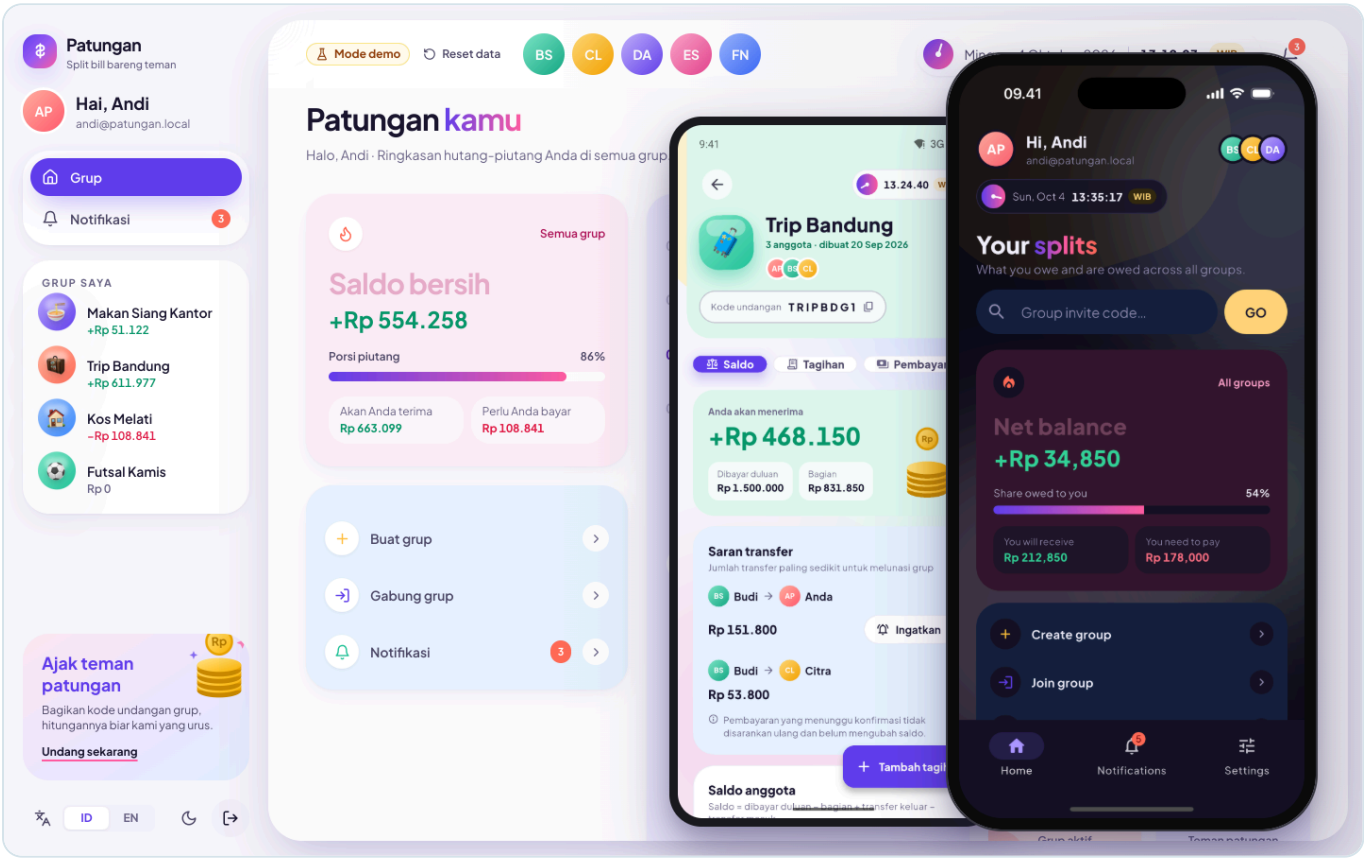
**PLATFORMS**

Web (desktop & mobile), Android, iOS

**LINKS**

Live demo (web): raihan-evanza-patungan.vercel.app/login

Go · Connect (gRPC) · Protocol Buffers + buf · PostgreSQL 16 · GORM · React 18 · TypeScript · Vite · Tailwind CSS · Redux Toolkit + RTK Query · Connect-ES · Flutter · Riverpod · gRPC Dart · Vision / ML Kit · Docker · GitHub Actions · Vercel




## OVERVIEW

After a group dinner or a trip, working out who owes whom usually turns into a mess in the group chat. Tax and service charge get split roughly, rupiah get lost to rounding, and chasing friends for money feels awkward.

Patungan records every bill inside a group and splits it equally, by item (with tax, service charge, and discounts), or by exact amounts. Everyone's balance is reduced to as few suggested transfers as possible. A payment only changes balances once the recipient confirms the money arrived, and people who haven't paid get automatic reminders.

The result is one Go backend with a gRPC (Connect) API, shared by a React web dashboard and a Flutter app for Android and iOS. The web app can be tried right away on Vercel through a demo mode that runs entirely in the browser.

## KEY FEATURES

### Three ways to split

Equal (leftover rupiah handed out one by one, so shares differ by at most Rp1), by item, or exact amounts that must add up to the total.

### Itemized with tax, service, discount

Each item is split among the people who had it, then tax, service charge, and discount are allocated in proportion to each person's subtotal. Quick buttons fill in 10%/11% tax and 5%/10% service.

### Server-side preview

While the form is filled in, the web app calls the PreviewSplit RPC and shows each person's share. The numbers come from the same code that runs on save, so they never differ.

### Balances and suggested transfers

Each member's net balance (paid up front − share + sent − received) becomes at most n−1 transfers. Pay and Remind buttons sit right on each suggested transfer.

### Payment confirmation

The sender records a payment as Pending; only the recipient can Confirm or Reject it (with a reason). The sender can cancel while it is still pending.

### Reminders and notifications

Creditors can send a manual reminder with a 12-hour cooldown, and a worker sends automatic reminders every 72 hours. Notification text is rendered in each reader's chosen language.

### Groups and invite codes

Friends join with an 8-character code that avoids look-alike letters. The owner can regenerate the code and remove members whose balance is zero.

### Receipt photos

A receipt photo can be attached to a bill (JPEG, PNG, or WebP, up to 5 MB). The file type is detected from its content, not from the label the client sends.

### Receipt scanning in the mobile app

The Flutter app reads the receipt on the device and fills in items, tax, service charge, and discount. The user still reviews the result before saving.

### Two languages and dark mode

Indonesian/English UI, including backend error messages that follow the Accept-Language header. Dark mode, responsive down to phone width, and animations turn off when the system asks for reduced motion.

## ARCHITECTURE & DATA FLOW

**Web React**
Connect-ES client generated from the proto, cached by RTK Query

→

**Flutter (Android & iOS)**
gRPC Dart client from the same proto, on-device receipt OCR

→

**Go backend (Connect)**
Interceptor → handler → usecase → split & balance packages

→

**PostgreSQL 16**
Amounts as BIGINT rupiah, plus receipt photo storage

- One backend port serves gRPC (HTTP/2), gRPC-Web, and Connect (JSON over HTTP/1.1), so neither the browser nor the mobile app needs a proxy.
- Interceptors handle logging, language, JWT auth, and turn domain errors into gRPC codes with ErrorInfo/BadRequest details.
- Business rules live in the usecase layer; money math lives in pure split and balance packages (no I/O), which keeps them easy to test. Notifications go out after the transaction commits, without holding up the request.
- In the Vercel demo, the Connect transport is swapped for an in-memory router: requests are still protobuf messages, but they are answered in the browser by a TypeScript port of the backend rules.

## STACK BY LAYER

| | |
|---|---|
| **API** | Go · Connect (gRPC, gRPC-Web, Connect) · Protocol Buffers · buf (lint STANDARD) · JWT + bcrypt · go-i18n · slog |
| **Data** | PostgreSQL 16 · GORM · SELECT … FOR UPDATE per grup |
| **Web** | React 18 · TypeScript · Vite · Tailwind CSS · Redux Toolkit + RTK Query · Connect-ES · React Router · react-hook-form + yup |
| **Mobile** | Flutter · Riverpod · go_router · gRPC Dart · flutter_secure_storage · Vision (iOS) · ML Kit (Android) |
| **Testing** | Testify · GoMock · Property test · E2E dengan Postgres asli · Race detector · Vitest · flutter_test + integration test |
| **Ops** | Docker multi-stage (±44 MB, non-root) · GitHub Actions · Vercel (demo web) |

## TECHNICAL DECISIONS

1. **Largest remainder for rounding**
   Every amount is stored as int64 rupiah, never as a float. Tax, service, and discount are split proportionally: each share is rounded down, then the leftover rupiah go to the largest fractions (using 128-bit multiplication to avoid overflow). Shares always add up to the total, which a property test checks against 2,000 random inputs.

2. **Debt simplification**
   Instead of one transfer per bill, the group's net balances are matched greedily: the largest debtor pays the largest creditor until everything is zero. That gives at most n−1 transfers, and payments still awaiting confirmation are not suggested again.

3. **The recipient confirms**
   A sender can mis-tap or a transfer can fail, so balances only change once the recipient confirms. Money operations in a group are locked with SELECT … FOR UPDATE and status changes check the previous status; an E2E test fires 5 full payments at once and only 1 goes through.

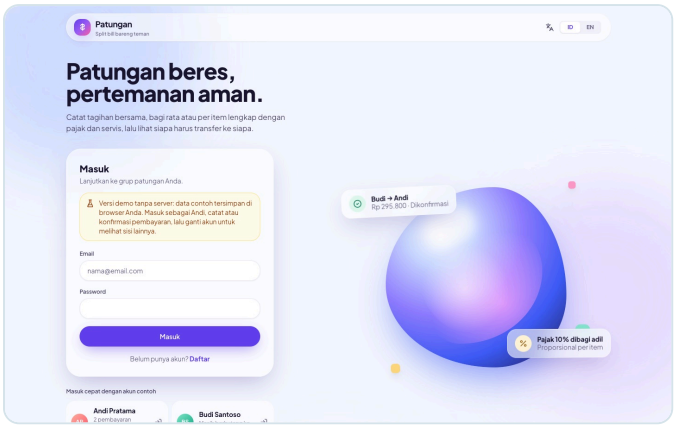
4. **One protobuf contract for web and Flutter**
   The API contract is written once in .proto files and checked by buf lint. The TypeScript (Connect-ES) and Dart (gRPC) clients are generated from the same files, so API changes surface at compile time instead of at runtime.

5. **On-device receipt OCR**
   Receipts are read on the phone through a platform channel: Vision on iOS and ML Kit on Android. No API cost, works offline, and photos are never sent to a third-party service; a Dart parser then merges aligned text into items and prices.

6. **Serverless demo mode on Vercel**
   So the app can be tried without paying for a server and database, the demo build swaps the Connect transport for createRouterTransport, which runs a port of the backend rules in the browser. Sample data lives in each visitor's localStorage, and the regular build ships none of the demo code.

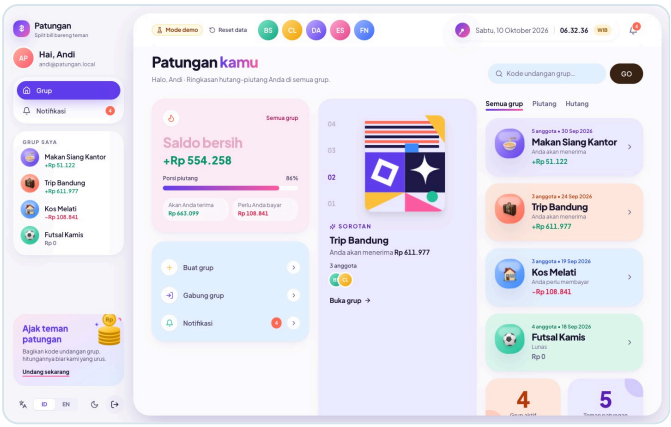
## HOW TO USE IT

These steps follow the same order as the tutorial video.



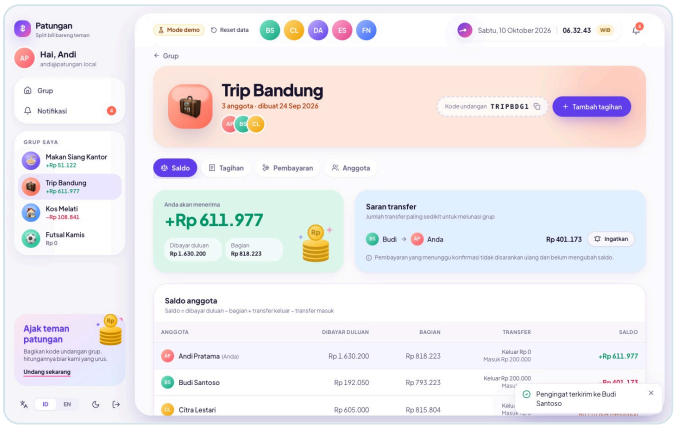

### 1 Sign in with a demo account

Open the demo and, under "Quick sign-in with a sample account", click Andi Pratama. No need to type a password; every account uses Password123!.



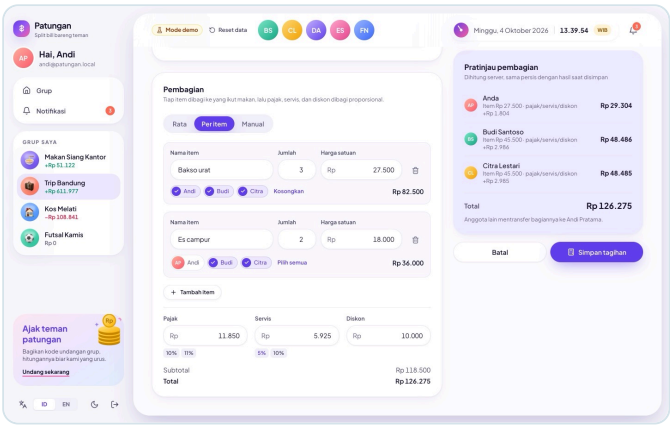

### 2 Read the dashboard

The dashboard shows Andi's net balance across all groups, how much he will receive and needs to pay, and each group with its own balance.



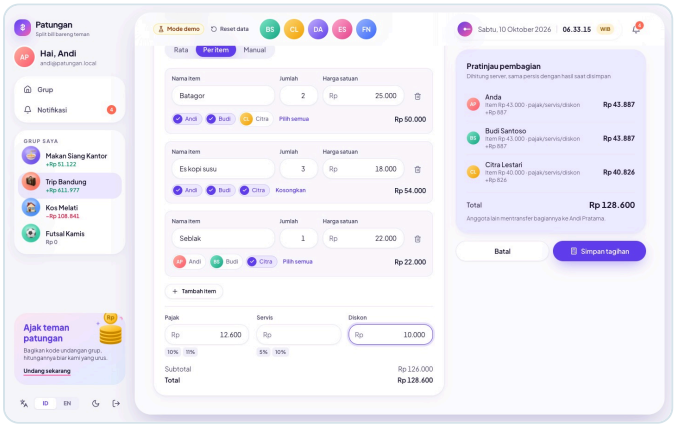

### 3 Open a group and its suggested transfers

Click the Trip Bandung group. The Balances tab shows the suggested transfer (Budi → Andi) with a Remind button, and every member's balance in the table below.



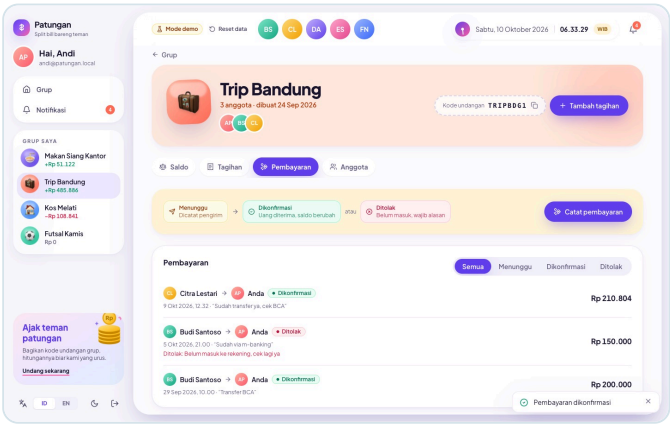

### 4 Record an itemized bill

Click Add bill, enter a title, and choose the Per item split. Fill in each item's name, quantity, and price, pick who shares it, then click Tax 10% and enter a discount.



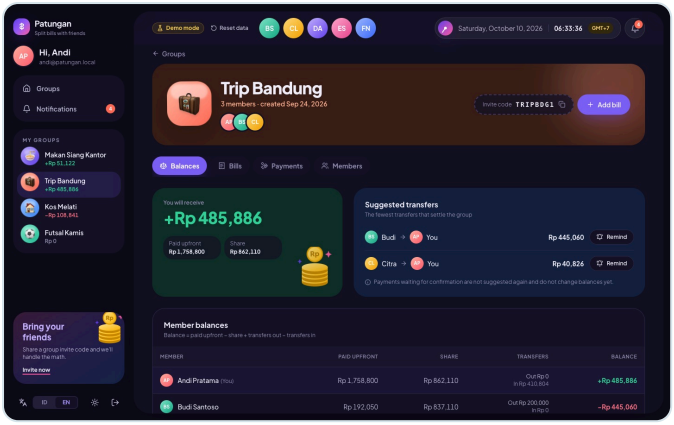

### 5 Check the preview, then save

The Split preview panel immediately shows each person's share, including their part of the tax and discount, and the total always matches. Click Save bill to open the bill details.




### 6 Confirm an incoming payment

Go back to the group and open the Payments tab. Citra's payment is Pending; click Confirm and approve it in the dialog, and the group balance updates.




### 7 Switch language and dark mode

At the bottom of the sidebar, choose EN for English and click the moon icon for dark mode. The Reset data button in the header restores the sample data.

## LIMITATIONS & NOTES

- The web demo runs without a backend: sign-in uses a demo token (not a JWT), data only exists in each visitor's browser, and there is no automatic reminder worker.
- The Go backend and PostgreSQL are not publicly hosted yet; the full version runs locally with Docker.
- Push notifications to phones (FCM) are not done yet; the mobile app uses in-app notifications with polling. The push provider interface is already in place.
- Deliberately out of scope: multiple currencies, a payment gateway, and editing bills (delete and recreate instead).
- Receipt scan results are OCR guesses, so the user still has to check them. JWTs last 7 days with no refresh token.

## WHAT'S NEXT

- FCM push through a Firebase project, including cleanup of invalid device tokens.
- Refresh tokens for the mobile app.
- Hosting the backend and PostgreSQL (e.g. Fly.io or Railway) so the demo can use the real API.